

Data Structures as Classes, and Why We Count Operations

Programming and Data Structures — Week 1, Lecture 4

Dr. Innocent Nyalala

From Shape to Stack

```
1 class Stack:
2     """Last-in, first-out collection."""
3
4     def __init__(self):
5         self._items = []           # hidden: nothing outside touches this list directly
6
7     def push(self, item):
8         self._items.append(item)   # add to the end
9
10    def pop(self):
11        return self._items.pop()    # remove and return from the end
12
13    def is_empty(self):
14        return len(self._items) == 0
```

Everything from Lectures 1 to 3 — a constructor, hidden state, methods that enforce correctness — is exactly the pattern every data structure in this course follows, starting next week with arrays.

Tracing it

```
s = Stack()
s.push(1)
s.push(2)
```

```
print(s.pop())      # 2 - the most recently pushed item comes back first
print(s.is_empty()) # False - one item, "1", is still inside
```

`_items` is a Python list, but nothing outside `Stack` is supposed to know that. A caller only ever sees `push`, `pop`, and `is_empty` — the internal representation could be swapped for a linked list next week and no code that *uses* `Stack` would need to change. That swappability is why we hide `_items` behind methods instead of just using a bare list everywhere.

Why “counting operations,” not “using a stopwatch”

```
import time

def linear_search(items, target):
    for i, x in enumerate(items):
        if x == target:
            return i
    return -1

data = list(range(1_000_000))
start = time.perf_counter()
linear_search(data, 999_999)
print(time.perf_counter() - start)
```

Run this on a laptop: some number of milliseconds. Run it on a phone: a different number. Run it after a Python upgrade: a third number. The stopwatch answers “how fast, on this machine, today” — a fact about the hardware, not about `linear_search` itself.

What actually stays true across machines

Input size	Comparisons (worst case)
1,000	1,000
1,000,000	1,000,000
1,000,000,000	1,000,000,000

However fast or slow one comparison `x == target` is, `linear_search` in the worst case performs it once for every element. Double the list, and in the worst case you double the comparisons — regardless of what machine you run it on. **That growth rate**, not the absolute time, is what Big-O describes.

Naming the growth rate

n	$O(n)$ linear search	$O(\log n)$ binary search
1,000	1,000	~10
1,000,000	1,000,000	~20
1,000,000,000	1,000,000,000	~30

We write this as $O(n)$ — “order n ” — meaning the cost grows in direct proportion to the input size n . Binary search on a *sorted* list, which you will meet in a few weeks, is $O(\log n)$: doubling the input adds only one more comparison, not one thousand more. This is why the distinction matters in practice, not just in theory: at a billion elements, one algorithm does thirty comparisons and the other does a billion.

Best, worst, and average case

`linear_search` finding the target in the very first slot does 1 comparison — its **best case**. Finding it last, or not at all, does n — its **worst case**. Big-O notation by convention describes the worst case unless stated otherwise, because that is the guarantee you can actually rely on.

Exercise (bring to lab)

Write `count_comparisons(items, target)` that behaves like `linear_search` but returns the *number of comparisons made*, not the index. Run it on lists of length 10, 100, and 1,000 with the target absent every time. Confirm the comparison count scales linearly with the list length.

Where this course goes next

Week 2 begins with arrays and dynamic arrays — the first data structure built as a class in this course, and the first place we will measure cost formally rather than informally.